

СТУДЕНТТІҢ ӨЗІНДІК ЖҰМЫСЫ №2

Пәні: Базы данных и разработка приложений

Тақырыбы: Дәрігерлердің жұмыс кестесі мен қабылдау жазбасы

Орындаған: Туменова Ғалия Аянқызы

Кіріспе

Бұл жұмыста медициналық мекемедегі дәрігерлердің жұмыс кестесін жүргізу және пациенттерді қабылдауға жазу процесіне арналған деректер қоры жобаланды. Жүйе дәрігер, мамандық, пациент, кабинет, жұмыс уақыты, қабылдау жазбасы, қызмет және төлем туралы мәліметтерді сақтайды.

Жұмыстың негізгі мақсаты – нормализация қағидаларын қолдану арқылы деректердің қайталануын азайту, процедуралар, функциялар, триггерлер және транзакциялар көмегімен деректер тұтастығын қамтамасыз ету.

1. Пәндік облыс сипаттамасы

Жүйеде пациент дәрігердің бос уақытына жазылады. Әр дәрігер белгілі бір мамандыққа жатады және белгілі кабинетте қабылдау жүргізе алады. Бір қабылдауда бір немесе бірнеше медициналық қызмет көрсетілуі мүмкін. Қабылдаудың статусы planned, completed, cancelled мәндерінің бірін қабылдайды. Төлем қабылдау аяқталғаннан кейін енгізіледі.

Объект	Сақталатын мәліметтер
Дәрігерлер	аты-жөні, мамандығы, телефоны, email
Пациенттер	аты-жөні, туған күні, телефон
Кесте	дәрігер, күн, басталу және аяқталу уақыты, кабинет
Қабылдау жазбасы	пациент, дәрігер, кесте, статус, ескерту
Қызметтер	қызмет атауы және бағасы
Төлемдер	төлем сомасы, әдісі, күні

2. Нормализация

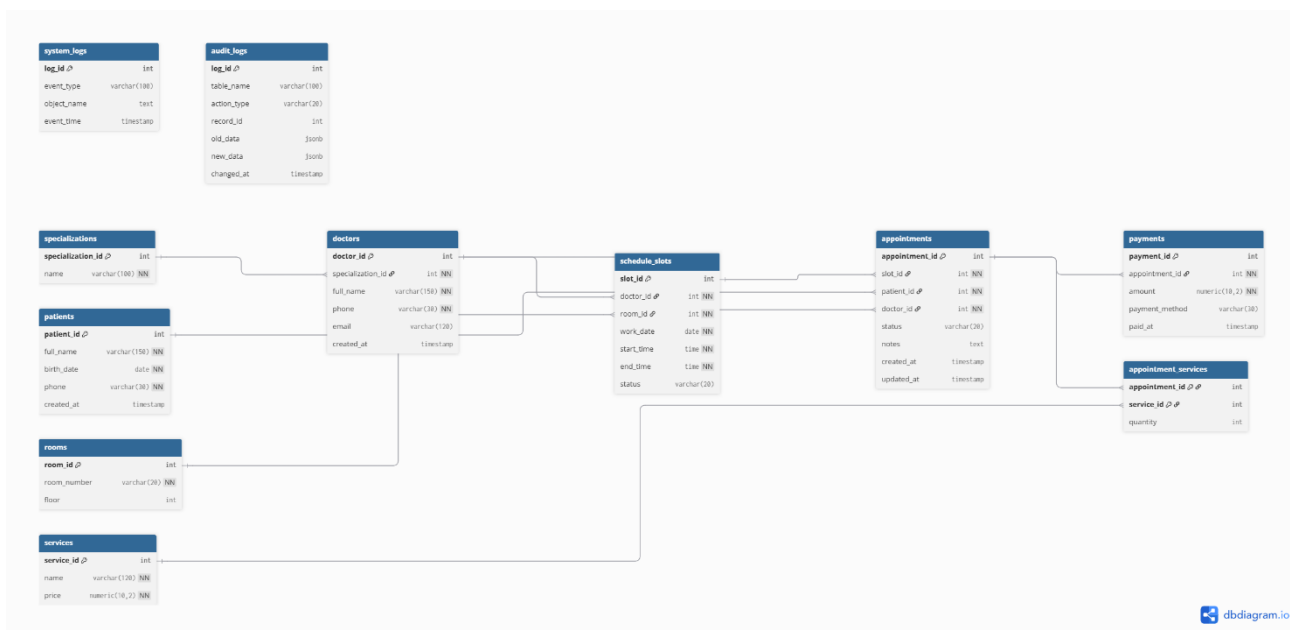
Бастапқыда барлық ақпарат бір үлкен кестеде сақталады: doctor_name, specialization, patient_name, appointment_date, start_time, end_time, room, services, total_price, payment_method. Мұндай кестеде дәрігер мен пациент туралы мәліметтер бірнеше рет қайталанатын, ал services бағанында бірнеше қызмет бір ұяшықта сақталуы мүмкін. Бұл қалыпты емес формаға жатады.

Нормалды форма	Мәселе	Жасалған өзгеріс	Нәтиже
1NF	Бір ұяшықта бірнеше мән болды: services = консультация, УДЗ	Қызметтер services кестесіне, байланыс appointment_services кестесіне шығарылды	Әр бағанда бір ғана мән сақталады
2NF	Құрама байланыс кестесінде қызмет атауы appointment_id-ға тәуелді емес	appointment_services кестесінде тек appointment_id, service_id, quantity қалдырылды	Ішінара тәуелділік жойылды
3NF	Мамандық атауы дәрігер арқылы, кабинет нөмірі кесте арқылы қайталануы мүмкін	specializations және rooms жеке кестеге бөлінді	Транзитивті тәуелділік азайды
BCNF	Әр детерминант кандидат кілт болуы керек	doctor_id + work_date + start_time және room_id + work_date + start_time үшін UNIQUE шектеулер қойылды	Бір уақытта бір дәрігер немесе бір кабинет екі рет қолданылмайды

3. Нормализациядан кейінгі кестелер

Кесте	Негізгі өрістер
specializations	specialization_id PK, name UNIQUE
doctors	doctor_id PK, specialization_id FK, full_name, phone, email UNIQUE
patients	patient_id PK, full_name, birth_date, phone UNIQUE
rooms	room_id PK, room_number UNIQUE, floor
schedule_slots	slot_id PK, doctor_id FK, room_id FK, work_date, start_time, end_time, status
appointments	appointment_id PK, slot_id FK UNIQUE, patient_id FK, doctor_id FK, status, notes
services	service_id PK, name UNIQUE, price
appointment_services	appointment_id FK, service_id FK, quantity, PK(appointment_id, service_id)

Кесте	Herізгі өпісrep
payments	payment_id PK, appointment_id FK, amount, payment_method, paid_at
audit_logs	log_id PK, table_name, action_type, record_id, old_data, new_data, changed_at



Дәрігерлердің жұмыс кестесі мен қабылдау жазбасы деректер қорының ER диаграммасы. Диаграммада кестелер арасындағы 1:N және N:M байланыстары көрсетілген.

4. Процедуралар мен функциялар

4 процедура және 4 функция құрылды.

Атауы	Түрі	Қызметі
add_patient	процедура	жаңа пациент қосады
add_schedule_slot	процедура	дәрігердің бос уақытын қосады
make_appointment	процедура	пациентті қабылдауға жазады
add_payment	процедура	төлем енгізеді және қабылдауды аяқтайды
get_appointment_total	функция	қабылдаудың жалпы сомасын есептейді
get_patient_visit_count	функция	пациенттің қабылдау санын шығарады

Атауы	Түрі	Қызметі
get_doctor_appointments_count	функция	дәрігердің қабылдау санын шығарады
get_free_slots	функция	дәрігердің бос уақыттарын шығарады

5. Триггерлер

Триггер	Деңгейі	Міндеті
trg_prevent_double_booking	жол деңгейі, BEFORE	бір уақытқа екі пациент жазылмауын тексереді
trg_update_appointment_time	жол деңгейі, BEFORE	қабылдау өзгергенде updated_at уақытын жаңартады
trg_log_appointments	лог-триггер, AFTER	INSERT, UPDATE, DELETE әрекеттерін audit_logs кестесіне жазады
trg_statement_appointments	оператор деңгейі, AFTER	оператор орындалғанын журналға белгілейді
trg_instead_insert_free_slot	INSTEAD OF	көрініс арқылы енгізілген бос уақытты негізгі кестеге жазады
trg_system_ddl_audit	жүйелік event trigger	DDL командаларды system_logs кестесіне тіркейді

6. Тестілеу нәтижесі

Команда	Мақсаты	Күтілетін нәтиже
CALL add_patient(...)	Жаңа пациент қосу	patients кестесінде жаңа жол пайда болады
CALL add_schedule_slot(...)	Дәрігердің бос уақытын қосу	schedule_slots кестесінде free статусы бар уақыт пайда болады
CALL make_appointment(...)	Пациентті қабылдауға жазу	appointments кестесіне жазба қосылады, slot status = booked
SELECT get_appointment_total(1)	Қабылдаудың жалпы сомасын есептеу	Қызметтер бағасының қосындысы шығады
CALL add_payment(1,'kaspi')	Төлем енгізу	payments кестесіне төлем түседі, қабылдау completed болады

7. Транзакциялар

Транзакцияда BEGIN арқылы операция басталады, SAVEPOINT арқылы аралық сақтау нүктесі жасалады. Егер қате операция орындалса, ROLLBACK TO SAVEPOINT арқылы тек сол қатеден кейінгі өзгерістер қайтарылады. Дұрыс мәлімет енгізілген соң COMMIT арқылы барлық өзгеріс сақталады.

ACID қағидасы	Бұл жұмыста көрінісі
Atomicity	Қате болса ROLLBACK жасалып, жартылай сақталған мәлімет қалмайды
Consistency	CHECK, FK, UNIQUE шектеулері деректерді дұрыс күйде ұстайды
Isolation	Транзакция орындалып жатқанда операциялар бір логикалық блок ретінде қаралады
Durability	COMMIT орындалғаннан кейін мәлімет базаға тұрақты сақталады

8. SQL кодының негізгі үзіндісі

-- Independent work 2: Дәрігерлердің жұмыс кестесі мен қабылдау жазбасы

```
DROP SCHEMA IF EXISTS clinic_schedule CASCADE;
```

```
CREATE SCHEMA clinic_schedule;
```

```
SET search_path TO clinic_schedule;
```

```
CREATE TABLE specializations (  
    specialization_id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL UNIQUE  
);
```

```
CREATE TABLE doctors (  
    doctor_id SERIAL PRIMARY KEY,  
    specialization_id INT NOT NULL REFERENCES specializations(specialization_id)  
,  
    full_name VARCHAR(150) NOT NULL,  
    phone VARCHAR(30) NOT NULL,  
    email VARCHAR(120) UNIQUE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE patients (  
    patient_id SERIAL PRIMARY KEY,  
    full_name VARCHAR(150) NOT NULL,  
    birth_date DATE NOT NULL,  
    phone VARCHAR(30) NOT NULL UNIQUE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```

CREATE TABLE rooms (
    room_id SERIAL PRIMARY KEY,
    room_number VARCHAR(20) NOT NULL UNIQUE,
    floor INT CHECK (floor >= 1)
);

CREATE TABLE schedule_slots (
    slot_id SERIAL PRIMARY KEY,
    doctor_id INT NOT NULL REFERENCES doctors(doctor_id) ON DELETE CASCADE,
    room_id INT NOT NULL REFERENCES rooms(room_id),
    work_date DATE NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    status VARCHAR(20) DEFAULT 'free' CHECK (status IN ('free', 'booked', 'closed'
)),
    CHECK (end_time > start_time),
    UNIQUE (doctor_id, work_date, start_time),
    UNIQUE (room_id, work_date, start_time)
);

CREATE TABLE appointments (
    appointment_id SERIAL PRIMARY KEY,
    slot_id INT NOT NULL UNIQUE REFERENCES schedule_slots(slot_id),
    patient_id INT NOT NULL REFERENCES patients(patient_id),
    doctor_id INT NOT NULL REFERENCES doctors(doctor_id),
    status VARCHAR(20) DEFAULT 'planned' CHECK (status IN ('planned', 'completed'
, 'cancelled')),
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE services (
    service_id SERIAL PRIMARY KEY,
    name VARCHAR(120) NOT NULL UNIQUE,
    price NUMERIC(10,2) NOT NULL CHECK (price >= 0)
);

CREATE TABLE appointment_services (
    appointment_id INT REFERENCES appointments(appointment_id) ON DELETE CASCADE
,
    service_id INT REFERENCES services(service_id),
    quantity INT DEFAULT 1 CHECK (quantity > 0),
    PRIMARY KEY (appointment_id, service_id)
);

CREATE TABLE payments (
    payment_id SERIAL PRIMARY KEY,
    appointment_id INT NOT NULL REFERENCES appointments(appointment_id),
    amount NUMERIC(10,2) NOT NULL CHECK (amount >= 0),
    payment_method VARCHAR(30) CHECK (payment_method IN ('cash', 'card', 'kaspi'))

```

```

    ,
    paid_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE audit_logs (
    log_id SERIAL PRIMARY KEY,
    table_name VARCHAR(100),
    action_type VARCHAR(20),
    record_id INT,
    old_data JSONB,
    new_data JSONB,
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE system_logs (
    log_id SERIAL PRIMARY KEY,
    event_type VARCHAR(100),
    object_name TEXT,
    event_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO specializations(name) VALUES
('Терапевт'), ('Кардиолог'), ('Стоматолог'), ('Педиатр');

INSERT INTO doctors(specialization_id, full_name, phone, email) VALUES
(1, 'Айдаров Нұрлан Серікұлы', '87010000001', 'n.aidarov@clinic.kz'),
(2, 'Сәрсенова Әлия Ерланқызы', '87010000002', 'a.sarsenova@clinic.kz'),
(3, 'Ким Денис Викторович', '87010000003', 'd.kim@clinic.kz');

INSERT INTO patients(full_name, birth_date, phone) VALUES
('Туменова Галия Аянқызы', '2005-05-10', '87075550000'),
('Омаров Еркебұлан Ержанұлы', '2000-02-15', '87075550001'),
('Садыкова Айша Мұратқызы', '2010-09-21', '87075550002');

INSERT INTO rooms(room_number, floor) VALUES ('101',1), ('205',2), ('307',3);

INSERT INTO schedule_slots(doctor_id, room_id, work_date, start_time, end_time)
VALUES
(1,1, '2026-04-27', '09:00', '09:30'),
(1,1, '2026-04-27', '09:30', '10:00'),
(2,2, '2026-04-27', '10:00', '10:30'),
(3,3, '2026-04-28', '11:00', '11:30');

INSERT INTO services(name, price) VALUES
('Алғашқы консультация', 7000), ('Қайта қабылдау', 5000), ('ЭКГ', 4000), ('Стоматологиялық тексеріс', 8000);

CREATE OR REPLACE PROCEDURE add_patient(p_full_name VARCHAR, p_birth_date DATE,
p_phone VARCHAR)
LANGUAGE plpgsql AS $$
BEGIN

```

```

    INSERT INTO patients(full_name, birth_date, phone) VALUES (p_full_name, p_birth_date, p_phone);
END; $$;

```

```

CREATE OR REPLACE PROCEDURE add_schedule_slot(p_doctor_id INT, p_room_id INT, p_work_date DATE, p_start_time TIME, p_end_time TIME)
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO schedule_slots(doctor_id, room_id, work_date, start_time, end_time)
    VALUES (p_doctor_id, p_room_id, p_work_date, p_start_time, p_end_time);
END; $$;

```

```

CREATE OR REPLACE PROCEDURE make_appointment(p_slot_id INT, p_patient_id INT, p_notes TEXT)
LANGUAGE plpgsql AS $$
DECLARE v_doctor_id INT; v_status VARCHAR(20);
BEGIN
    SELECT doctor_id, status INTO v_doctor_id, v_status FROM schedule_slots WHERE slot_id = p_slot_id;
    IF v_doctor_id IS NULL THEN RAISE EXCEPTION 'Мұндай кесте уақыты табылмады';
END IF;
    IF v_status <> 'free' THEN RAISE EXCEPTION 'Бұл уақыт бос емес'; END IF;
    INSERT INTO appointments(slot_id, patient_id, doctor_id, notes) VALUES (p_slot_id, p_patient_id, v_doctor_id, p_notes);
    UPDATE schedule_slots SET status = 'booked' WHERE slot_id = p_slot_id;
END; $$;

```

```

CREATE OR REPLACE PROCEDURE add_payment(p_appointment_id INT, p_payment_method VARCHAR)
LANGUAGE plpgsql AS $$
DECLARE v_amount NUMERIC(10,2);
BEGIN
    SELECT COALESCE(SUM(s.price * aps.quantity), 0) INTO v_amount
    FROM appointment_services aps JOIN services s ON s.service_id = aps.service_id
    WHERE aps.appointment_id = p_appointment_id;
    INSERT INTO payments(appointment_id, amount, payment_method) VALUES (p_appointment_id, v_amount, p_payment_method);
    UPDATE appointments SET status = 'completed' WHERE appointment_id = p_appointment_id;
END; $$;

```

```

CREATE OR REPLACE FUNCTION get_appointment_total(p_appointment_id INT) RETURNS NUMERIC(10,2)
LANGUAGE plpgsql AS $$
DECLARE v_total NUMERIC(10,2);
BEGIN
    SELECT COALESCE(SUM(s.price * aps.quantity), 0) INTO v_total
    FROM appointment_services aps JOIN services s ON s.service_id = aps.service_id
    WHERE aps.appointment_id = p_appointment_id;

```



```

        RETURN v_total;
END; $$;

CREATE OR REPLACE FUNCTION get_patient_visit_count(p_patient_id INT) RETURNS INT
LANGUAGE plpgsql AS $$
DECLARE v_count INT;
BEGIN
    SELECT COUNT(*) INTO v_count FROM appointments WHERE patient_id = p_patient_
id;
    RETURN v_count;
END;
$$;

```

Tables (11)

- appointment_services
- appointments
- audit_logs
- doctors
- patients
- payments
- rooms
- schedule_slots
- services
- specializations
- system_logs
- Trigger Functions

clinic_schedule схемасында құрылған кестелер тізімі.

```

302 SELECT * FROM patients;
303 SELECT * FROM doctors;
304 SELECT * FROM appointments;

```

Data Output Messages Notifications

Showing rows: 1 to 2 of 1

	appointment_id integer	slot_id integer	patient_id integer	doctor_id integer	status character varying (20)	notes text	created_at timestamp without time zone	updated_at timestamp without time zone
1	1	1	1	1	completed	Алғашқы қабылдау	2026-05-02 00:33:53.97282	2026-05-02 00:33:53.97282
2	2	2	2	1	planned	Кардиологқа жазы...	2026-05-02 00:33:53.97282	2026-05-02 00:33:53.97282

Кестелерге енгізілген бастапқы деректер нәтижесі.

```

306 CALL add_patient('Айгерім', '2002-05-10', '87011112233');
307
308 SELECT * FROM patients;

```

Data Output Messages Notifications

Showing rows: 1

	patient_id integer	full_name character varying (150)	birth_date date	phone character varying (30)	created_at timestamp without time zone
1	1	Туменова Галия Аянқызы	2005-05-10	87075550000	2026-05-02 00:30:41.881893
2	2	Омаров Еркебұлан Ержанұ...	2000-02-15	87075550001	2026-05-02 00:30:41.881893
3	3	Садықова Айша Мұратқызы	2010-09-21	87075550002	2026-05-02 00:30:41.881893
4	4	Нұрбекова Дана Қайратқы...	1999-01-12	87075550003	2026-05-02 00:33:53.97282
5	5	Айгерім	2002-05-10	87011112233	2026-05-02 01:17:13.671168

add_patient процедурасын орындау нәтижесі. Процедура арқылы жаңа пациент кестеге қосылды.

```

314 SELECT * FROM schedule_slots;
315
316 CALL make_appointment(4, 1, 'Кеңес алу');
317
318 SELECT * FROM appointments;

```

	appointment_id [PK] integer	slot_id integer	patient_id integer	doctor_id integer	status character varying (20)	notes text	created_at timestamp without time zone	updated_at timestamp without time zone
1	1	1	1	1	completed	Алғашқы қабылдау	2026-05-02 00:33:53.97282	2026-05-02 00:33:53.97282
2	2	2	2	1	planned	Кардиологқа жазы...	2026-05-02 00:33:53.97282	2026-05-02 00:33:53.97282
3	4	4	1	3	planned	Кеңес алу	2026-05-02 01:19:40.125519	2026-05-02 01:19:40.125519

make_appointment процедурасы арқылы пациент дәрігер қабылдауына жазылды.

```

321
322 SELECT get_patient_visit_count(1) AS patient_visits;
323
324 SELECT * FROM get_free_slots(1, '2026-04-27');

```

	patient_visits integer
1	2

get_patient_visit_count функциясының жұмысын тексеру нәтижесі. Нәтижеде *patient_visits* = 2 шықты, яғни 1-нөмірлі пациенттің қабылдау саны 2 екенін көрсетеді.

```

324 SELECT * FROM get_free_slots(2, '2026-04-27');

```

	slot_id integer	work_date date	start_time time without time zone	end_time time without time zone	room_number character varying
1	3	2026-04-27	10:00:00	10:30:00	205

get_free_slots функциясының нәтижесі. Бұл функция таңдалған дәрігердің көрсетілген күндегі бос уақыттарын шығарады.

```

328
329 SELECT * FROM audit_logs ORDER BY changed_at DESC;
330 SELECT * FROM system_logs ORDER BY event_time DESC;

```

	log_id [PK] integer	table_name character varying (100)	action_type character varying (20)	record_id integer	old_data jsonb
1	9	appointments	INSERT	4	[null]
2	10	appointments	INSERT_STATEMENT	[null]	[null]
3	3	appointments	UPDATE	1	{'notes': 'Алғашқы қабылдау', 'status': 'planned', 'slot_id': 1, 'doctor_id': 1, 'created_at': '2026-05-02T00:33:53.97282'}
4	4	appointments	UPDATE_STATEMENT	[null]	[null]

Триггерлердің жұмыс нәтижесі. *audit_logs* кестесінде *appointments* кестесіне жасалған *INSERT* және *UPDATE* әрекеттері автоматты түрде сақталған.

335

BEGIN;

336

CALL make_appointment(3, 3, 'Тест қабылдау');

337

ROLLBACK;

338

339

SELECT * FROM appointments;

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 3

Page No: 1 of 1

appointment_id [PK] integer	slot_id integer	patient_id integer	doctor_id integer	status character varying (20)	notes text	created_at timestamp without time zone	updated_at timestamp without time zone
1	1	1	1	completed	Алғашқы қабылдау	2026-05-02 00:33:53.97282	2026-05-02 00:33:53.97282
2	2	2	2	planned	Кардиологқа жазы...	2026-05-02 00:33:53.97282	2026-05-02 00:33:53.97282
3	4	4	1	planned	Кеңес алу	2026-05-02 01:19:40.125519	2026-05-02 01:19:40.125519

✓ Successfully run. Total query runtime: 150 msec. 3 rows affected.

ROLLBACK арқылы транзакцияны кері қайтару нәтижесі. Тест ретінде қосылған қабылдау сақталмады, себебі транзакция ROLLBACK командасымен кері қайтарылды.

Қорытынды

Бұл жұмыста дәрігерлердің жұмыс кестесі мен қабылдау жазбасына арналған деректер қоры жобаланды. Алғашқы нормалды емес құрылым 1NF, 2NF, 3NF және BCNF талаптарына сәйкестендірілді. Нәтижесінде деректердің қайталануы азайып, дәрігер, пациент, қызмет және қабылдау мәліметтері бөлек кестелерде сақталды.

Процедуралар арқылы пациент қосу, кесте құру, қабылдауға жазу және төлем енгізу автоматтандырылды. Функциялар қабылдау сомасын, пациенттің келу санын, дәрігердің қабылдау санын және бос уақыттарды табуға мүмкіндік береді. Триггерлер деректердің қайталануын болдырмайды, өзгерістерді журналға жазады және көрініс арқылы енгізуді басқарады. Транзакциялар арқылы қате болған жағдайда өзгерістерді кері қайтару және дұрыс операцияларды сақтау көрсетілді.